



COS30018 - Intelligent System

Project: Stock Price Prediction System

Task C.5 - Machine Learning - Part 2

Student Name: Ngo Sy Vuong

Student ID: 105551480

Tutor: Nguyen Manh Toan

Tutorial Session: Wednesday 8:00 - 12:00

Semester: May - July 2026

Table of Contents

Content	Page
Cover Page	1
Table of Contents	2
<u>Environment Setup</u>	3
<u>1. Environment</u>	3
<u>2. Dependencies</u>	3
<u>3. Deployment</u>	3
<u>Executive Summary</u>	4
<u>Stock Prediction Version 5</u>	4
<u>1. Multistep Univariate Prediction</u>	4
<u>2. Single-step Multivariate Prediction</u>	6
<u>3. Multistep Multivariate Prediction</u>	8
<u>4. Experiments</u>	10

Environment Setup

To successfully replicate and run the stock price prediction model, please fulfill the following structural and software dependencies.

1. Environment

The system is built on an isolated virtual environment (venv) to ensure dependency abstraction and eliminate version conflicts with global system libraries:

- Core Runtime: **Python 3.11.x (64-bit)**
- Execution Interface: **Windows Command Prompt (CMD)**

2. Dependencies

tensorflow	Core Deep Learning framework used to compile structure and execute the LSTM network
yfinance	Automated financial data retrieval tool used to fetch historical structural data for CBA.AX via the Yahoo Finance API
scikit-learn	Provides data preprocessing mechanics
pandas	Sequential series concatenation, index cleaning and CSV file extraction
numpy	Low-level vectorized array manipulation, matrix dimension reshaping and numeric tensor optimization
matplotlib	Render visual evaluations
mplfinance	Specialized financial visualization extension built on Matplotlib
seaborn	Statistical data visualization framework built on top of Matplotlib

3. Deployment

To reconstruct this environment layout from scratch, run the following sequential commands within the target file directory: **(make sure you have installed Python 3.11)**

```
# Initialize the Python 3.11 Isolation layer
py -3.11 -m venv venv
```

```
# Activate the localized scripts block (Windows)
venv\Scripts\activate.bat
```

```
# Synchronize deployment prerequisites
pip install -r requirements.txt
```

Executive Summary

The purpose of Task C.5 is to automate and extend the inference capabilities of our system by creating a prediction module called `future_prediction.py`. Instead of being restricted to rigid, single-step ($t+1$) next-day forecasting, this module houses advanced wrapper functions that shift our model into an out-of-sample horizon forecaster without altering its underlying architecture.

Specifically, Task C.5 delivers a unified script that solves three unique functional paradigms:

- First, it creates a simple multivariate single-step prediction function that cleanly extracts a single future target price from multi-feature snapshots.
- Second, it implements a recursive, multistep prediction function that loops over a customizable horizon vector (k -days ahead) by continually feeding its own previous close predictions back into the model as input sequences.
- Third, it combines these concepts into a multivariate, multistep framework by dynamically updating the target Close price while cleanly carrying forward non-target features (Open, High, Low, Volume) at their last known state.

Ultimately, Task C.5 does not modify the structural weights or optimize the learning parameters established in Task C.4. It transforms the core prediction pipeline, combining these advanced recursive strategies into a separate file so that the main program (`stock_prediction.py`) can cleanly execute multi-day horizon projections and output real-time sequence evaluations.

Stock Prediction Version 5

1. Multistep Univariate Prediction

The function `predict_multistep_close` is designed to solve the multistep-ahead time-series prediction problem. To project a sequence of k -days into an unobserved future, this function implements an Iterative Forecasting Strategy.

In this paradigm, the model predicts the immediately following timestep, and this predicted value is dynamically appended to the input sequence vector for the subsequent prediction step ($t+2$), repeating until the horizon k is reached.

Upon initialization, the function secures data integrity by applying `np.copy()` to isolate the historical 60-day testing window (`last_window`) from upstream corruption. To accommodate the rigid multi dimensional constraints of the TensorFlow Keras API, a conditional check determines whether the incoming data layout is a flat 2D matrix, if verified, `np.expand_dims()` is executed to inject a dummy batch dimension at index position 0. This operation structurally transforms the matrix geometry from a native sequence layout, such as (60, 5) into an explicitly batched 3D tensor layout of (1, 60, 5) representing (batch_size, timesteps, features), thereby avoiding input alignment faults during network inference.

```
# model: LSTM / GRU / RNN
# last_window: The final 60-day window of scaled data from the test set
# k_days: The number of days into the future
# target_scaler: The MinMaxScaler object used for the Close price
# target_idx: The column index position of the Close price
def predict_multistep_close(model, last_window, k_days, target_scaler, target_idx):

    # First, create an independent copy of the starting 60-day historical window to avoid modifying the original
    # Then, check if the array is 2-dimensional (because TensorFlow's LSTM layers require a 3-dimensional input)
    # If 2-dimensional, transfer into 3-dimensional (for example, from (60,5) into (1,60,5))
    current_window = np.copy(last_window)
    if len(current_window.shape) == 2:
        current_window = np.expand_dims(current_window, axis=0)
```

Figure 1.1 - `predict_multistep_close` function part 1

Inside the primary execution loop, the model iteratively computes an out-of-sample prediction sequence across a customizable horizon parameter (`k_days`). For each step, a forward pass is executed via `model.predict()`. The output array is instantly unpacked using multi-dimensional array slicing at index `[0, 0]` to strip away outer tensor brackets `[]` and unwrap the raw scalar value, which is then preserved inside a tracking cache (`future_predictions`). (See Figure 1.1)

```
for i in range(k_days):

    # Passes the current 60-day matrix through the LSTM model to generate a prediction for the next day
    # Then extract the raw numerical scalar prediction value from the output array and saves it
    # Checking index [0,0] unwraps the raw float value (strips away the outer matrix brackets [])
    next_pred = model.predict(current_window)
    future_predictions.append(next_pred[0, 0])

    # Extracts a copy of the very last row (the most recent day) from the 60-day matrix sequence
    # (Index 0 enters the batch dimension, -1 grabs the most recent time-step, and : grabs all features)
    # Then overwrite the closing price in the template row with the most recent prediction
    # Since this is a close price prediction module, we preserve our non-target values (like volume or high/low metrics)
    # but overwrite the target cell with the most recent prediction
    next_timestep = np.copy(current_window[0, -1, :])
    next_timestep[target_idx] = next_pred[0, 0]

    # Shift the 60-day window forward by one day
    # current_window[:, 1:, :]: Drop the oldest day in the matrix (index 0), reducing the
    # lookback window temporarily from 60 days to 59 days
    # np.append(..., [[next_timestep]], axis=1): Attach the newly synthesized day onto the end of the timeline
    # along the time-axis (axis=1). This brings the window back to exactly 60 days
    current_window = np.append(current_window[:, 1:, :], [[next_timestep]], axis=1)

# Converts the gathered list of predictions back into a structured NumPy array and reshapes it into a single column
# (Scikit-Learn scalers require data inputs to be shaped (N, 1) to perform mathematical inverse operations)
# Then reverses the scaling normalization to convert the data back into raw currency values,
# collapses the array back down into a clean 1D list, and returns it
future_predictions = np.array(future_predictions).reshape(-1, 1)
return target_scaler.inverse_transform(future_predictions).flatten()
```

Figure 1.2 - `predict_multistep_univariate` function part 2

To simulate the progression of time, the function implements a `next_timestep` by making a deep copy of the most recent timestep in the matrix (`current_window[0, -1, :]`). While secondary, non-target environmental metrics (volume, opening, and high/low values) are kept static to preserve operational context, the target index (`target_idx`) is systematically overwritten with the newly inferred closing scalar. The lookback sequence is then advanced using a sliding-window shift operation: `current_window[:, 1:, :]` extracts a trimmed timeline from index 1 forward, effectively discarding the oldest chronological row and dropping the lookback length from 60 days to 59 days, while `np.append(..., axis=1)` attaches the newly engineered future template row back to the end of the tensor along the temporal axis. This continuously maintains an exact 60-day lookback window across every loop iteration. Once the execution loop concludes, the accumulated prediction cache is cast into a structured NumPy array and forced into a strict column layout (`.reshape(-1, 1)`) to satisfy the dimensionality requirements of the Scikit-Learn framework. Finally, the column matrix is passed through `inverse_transform()` to reverse the training normalization, collapsed into a flat 1D vector using `.flatten()`, and returned as a clear sequence of projected stock valuations. (See Figure 1.2)

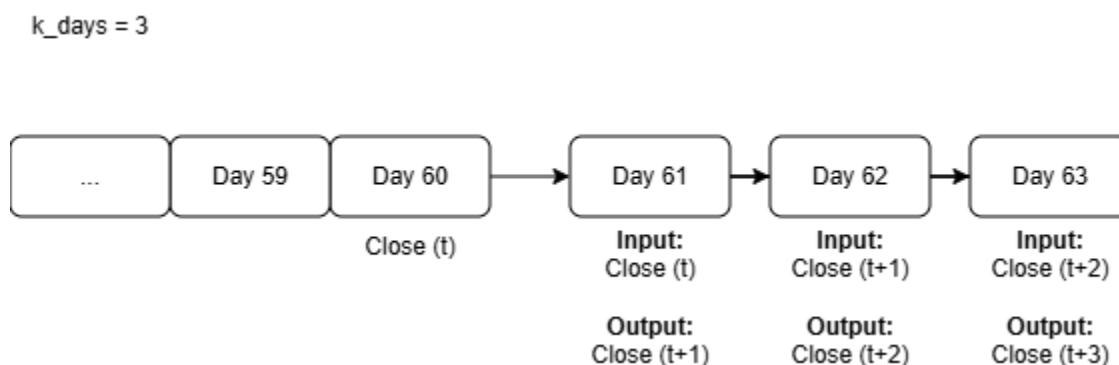


Figure 1.3 - Example flow of `predict_multistep_univariate` with `k_days = 3`

2. Single-step Multivariate Prediction

The second module component, implemented via the `predict_multivariate_single_step` function, serves as a specialized deterministic wrapper to execute single-step-ahead ($t+1$) regression tracking. This function takes the inputs including Open, High, Low, Close, and Volume variables to extract a closing price valuation for the immediate next trading day.

To initiate inference without causing structural execution failures, the function evaluates the dimensional depth of the incoming historical sequence block (`input_matrix`). Because a single, unbatched lookback slice extracted from the data preprocessing pipeline naturally retains a flat 2D matrix layout of 60 timesteps x 5 features, it fails to meet the strict tensor rank requirements of recurrent neural architectures. To resolve this geometric mismatch, `np.expand_dims()` is deployed to inject an explicit batch dimension at index position 0. This operation dynamically expands the array's geometric configuration into a 3D tensor layout of shape (1, 60, 5), which perfectly satisfies the strict Keras input convention. (See Figure 2.1)

```

# model: LSTM / GRU / RNN
# input_matrix: A 60-day window containing all 5 features
#                 (['Open', 'High', 'Low', 'Close', 'Volume'])
# target_scaler: The MinMaxScaler object used for the Close price
def predict_multivariate_single_step(model, input_matrix, target_scaler):

    # Same as predict_multistep_close, convert 2D to 3D
    if len(input_matrix.shape) == 2:
        input_matrix = np.expand_dims(input_matrix, axis=0)

    # Perform prediction (with all 5 variables)
    # Then reverse the normalization scaling, extracts the raw float number, and returns it
    prediction = model.predict(input_matrix)
    return target_scaler.inverse_transform(prediction)[0][0]

```

Figure 2.1 - `predict_multivariate_single_step` function

Once structural compatibility is achieved, the 3D multi-feature tensor is passed directly into the recurrent network for a single forward pass execution via `model.predict()`. The underlying architecture maps the non-linear interactions across all five technical market metrics over the 60-day historical timeline down to a single normalized scalar output. Because the network operates entirely within a compressed mathematical vector space established during the training normalization phase, this raw output is initially returned as an abstract decimal between 0 and 1, wrapped inside a nested multi dimensional array. (See Figure 2.1)

To transform this abstract mathematical coefficient into a practical financial metric, the prediction array is passed through the `inverse_transform()` function of the specific `target_scaler` instance assigned to the Close price column. Finally, array indexing using `[0][0]` is applied to strip away the outer matrix array brackets `[]`, extracting the raw floating-point scalar value. This returns an unscaled currency valuation representing the predicted closing price for day 61.

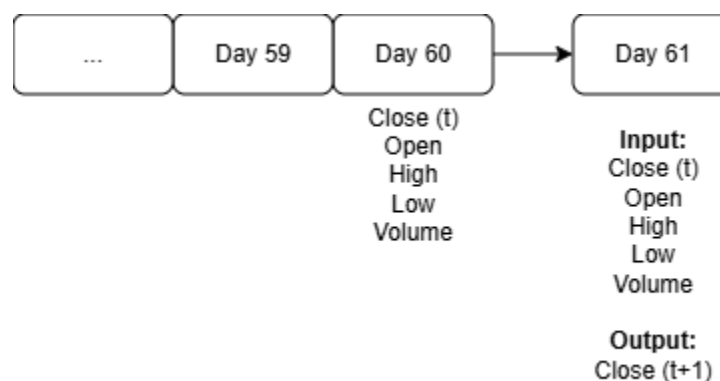


Figure 2.2 - Example flow of `predict_multivariate_single_step`

3. Multistep Multivariate Prediction

The final module component, implemented via the `predict_multivariate_multistep` function, merges multi-feature input patterns with an autoregressive, recursive loop to forecast a sequence of closing prices across a multi-day horizon (k-days ahead). This approach allows the system to bridge the gap between single-step output limitations and multi-day planning needs.

To ensure computational isolated execution, the function clones the initial historical tracking data (`last_window`) using `np.copy()`, creating an isolated matrix array (`current_window`). This preserves the integrity of your original testing dataset. Next, a structural conditional block checks the matrix dimensions. If the array is detected as a native 2D structure of shape (60, 5), `np.expand_dims()` is executed to append an explicit batch dimension at index position 0. This reshapes the matrix into a formal 3D tensor layout of shape (1, 60, 5), fully satisfying the strict batch layout rules required by Keras recurrent neural networks. (See Figure 3.1)

```
# model: LSTM / GRU / RNN
# last_window: The final 60-day window of scaled data from the test set
# k_days: The number of days into the future
# target_scaler: The MinMaxScaler object used for the Close price
# target_idx: The column index position of the Close price
def predict_multivariate_multistep(model, last_window, k_days, target_scaler, target_idx):

    # Create an independent copy
    current_window = np.copy(last_window)

    # Same as predict_multistep_close and predict_multivariate_single_step, convert 2D to 3D
    if len(current_window.shape) == 2:
        current_window = np.expand_dims(current_window, axis=0)

    # An empty list to store the outputs
    predictions = []

    # Same as predict_multistep_close
    for i in range(k_days):
        pred_scaled = model.predict(current_window)
        predictions.append(pred_scaled[0, 0])

        next_row = np.copy(current_window[0, -1, :])
        next_row[target_idx] = pred_scaled[0, 0]

        current_window = np.concatenate((current_window[:, 1:, :], [[next_row]]), axis=1)

    # Convert into shape (N,1) then reverses the scaling normalization to convert the data back into raw currency values
    predictions = target_scaler.inverse_transform(np.array(predictions).reshape(-1, 1))
    return predictions.flatten()
```

Figure 3.1 - `predict_multivariate_multistep` function

Inside the primary execution loop, the function manually manages the passage of time over the designated horizon (`k_days`). At each iteration, the multi-feature tensor passes through a `model.predict()`. The network evaluates the structural relationship between all five historical variables and generates a single scaled output. The function instantly extracts this value using multi-dimensional index coordinates at `[0, 0]`, stripping away the outer tensor brackets `[]` to unwrap the raw decimal scalar before storing it inside an accumulation list (`predictions`).

To advance the timeline to the next trading day without real-world observations, the function addresses the unobserved future feature dilemma by enforcing a static environmental assumption. First, `np.copy(current_window[0, -1, :])` isolates a flat row vector containing the 5 feature states from the most recent known step. While secondary non-target parameters (Open, High, Low, and Volume) are held completely frozen at their last known values to preserve the baseline market environment, the target column index (`target_idx`) is dynamically updated with the model's own newly inferred Close price prediction.

After that, the syntax `current_window[:, 1:, :]` drops the oldest day (index 0) from the timeline, reducing the window length temporarily from 60 days to 59 days. Then, `np.concatenate(..., axis=1)` takes the modified template row (`next_row`) and welds it directly onto the end of the tensor along the time axis (`axis=1`). This restores the lookup length to exactly 60 days for the next loop iteration. Once the horizon loop completes, the collected list of decimals is cast into a NumPy matrix and forced into a single-column orientation using `.reshape(-1, 1)` to align with Scikit-Learn data shape rules. Finally, `inverse_transform()` reverses the scaling normalization to restore the original currency valuations, while `.flatten()` collapses the matrix back into a clean 1D sequence array for direct reporting or visualization. (See Figure 3.1)

`k_days = 3`

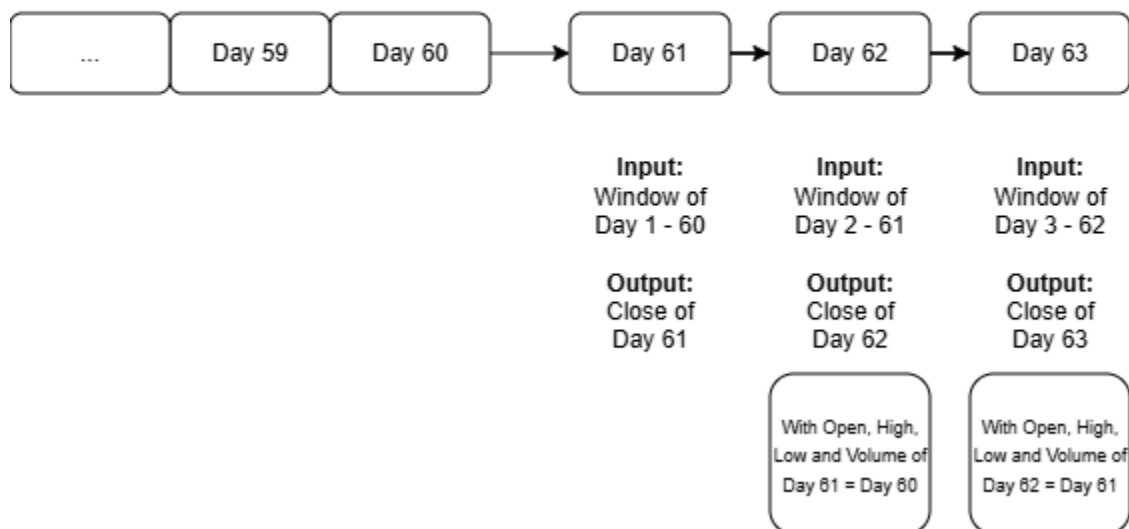


Figure 3.2 - Example flow of `predict_multivariate_multistep`

4. Experiments

After 3 run tests (See Figure 4.1, 4.2 and 4.3), the identical values observed across all three functions in the terminal output are a direct mathematical consequence of shared initial conditions and structural alignment in the recursive looping logic.

For the first predicted timestep ($t+1$), the single-step multivariate, the multistep univariate, and the multistep multivariate functions all execute a forward pass using the exact same historical lookback matrix containing 100% real-world data from Day 1 to Day 60. Because the input arrays are bitwise identical at this point, the underlying neural network weights generate the exact same scalar prediction of.

Furthermore, the absolute numerical parity maintained from Day 2 through Day 7 between the univariate and multivariate multistep functions occurs because both blocks of code handle the unobserved future identically under the hood. In both algorithms, tomorrow's input matrix row is engineered by taking a deep copy of the final known timestep, locking the non-target variables (Open, High, Low, Volume) at their last known values, and overwriting only the target closing price index with the model's own latest prediction. Since both functions end up sliding an identical matrix configuration of updated closing prices alongside frozen context features into the model at every single iteration, their mathematical trajectories match identically, demonstrating that both pipelines collapse into the exact same autoregressive prediction mechanism.

```
27/27 ----- 1s 23ms/step - loss: 0.0040
8/8 ----- 0s 33ms/step
1/1 ----- 0s 22ms/step
Single-step Multivariate Prediction: $110.36
1/1 ----- 0s 29ms/step
1/1 ----- 0s 29ms/step
1/1 ----- 0s 27ms/step
1/1 ----- 0s 23ms/step
1/1 ----- 0s 24ms/step
1/1 ----- 0s 27ms/step
1/1 ----- 0s 24ms/step
Multistep Univariate Prediction with (7 days ahead): [110.36 110.36 110.29 110.16 109.98 109.79 109.59]
1/1 ----- 0s 27ms/step
1/1 ----- 0s 26ms/step
1/1 ----- 0s 28ms/step
1/1 ----- 0s 30ms/step
1/1 ----- 0s 23ms/step
1/1 ----- 0s 29ms/step
1/1 ----- 0s 26ms/step
Multistep Multivariate Prediction with (7 days ahead): [110.36 110.36 110.29 110.16 109.98 109.79 109.59]
(venv) D:\COS30018\COS30018_Project>
```

Figure 4.1 - Run test 1

```

27/27 ----- 1s 52ms/step - loss: 0.0040
8/8 ----- 1s 52ms/step
1/1 ----- 0s 24ms/step
Single-step Multivariate Prediction: $113.19
1/1 ----- 0s 24ms/step
1/1 ----- 0s 30ms/step
1/1 ----- 0s 28ms/step
1/1 ----- 0s 28ms/step
1/1 ----- 0s 28ms/step
1/1 ----- 0s 22ms/step
1/1 ----- 0s 25ms/step
Multistep Univariate Prediction with (7 days ahead): [113.19 113.21 113.15 113.02 112.84 112.63 112.42]
1/1 ----- 0s 26ms/step
1/1 ----- 0s 25ms/step
1/1 ----- 0s 25ms/step
1/1 ----- 0s 24ms/step
1/1 ----- 0s 28ms/step
1/1 ----- 0s 24ms/step
1/1 ----- 0s 26ms/step
Multistep Multivariate Prediction with (7 days ahead): [113.19 113.21 113.15 113.02 112.84 112.63 112.42]
(venv) D:\COS30018\COS30018_Project>

```

Figure 4.2 - Run test 2

```

27/27 ----- 1s 34ms/step - loss: 0.0040
8/8 ----- 1s 54ms/step
1/1 ----- 0s 32ms/step
Single-step Multivariate Prediction: $108.36
1/1 ----- 0s 36ms/step
1/1 ----- 0s 34ms/step
1/1 ----- 0s 33ms/step
1/1 ----- 0s 32ms/step
1/1 ----- 0s 34ms/step
1/1 ----- 0s 35ms/step
1/1 ----- 0s 36ms/step
Multistep Univariate Prediction with (7 days ahead): [108.36 108.33 108.18 107.91 107.58 107.22 106.87]
1/1 ----- 0s 33ms/step
1/1 ----- 0s 36ms/step
1/1 ----- 0s 37ms/step
1/1 ----- 0s 36ms/step
1/1 ----- 0s 37ms/step
1/1 ----- 0s 38ms/step
1/1 ----- 0s 33ms/step
Multistep Multivariate Prediction with (7 days ahead): [108.36 108.33 108.18 107.91 107.58 107.22 106.87]
(venv) D:\COS30018\COS30018_Project>

```

Figure 4.3 - Run test 3